pfi

# Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

# SQL Joins and Time Series

WEEK 3 · DAY 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# From one table to many

Most desk questions need two tables or more: which sector each trade was in, how much was traded in investment grade, which benchmark bonds the book does not hold. Each answer is a join.

A join on a key that repeats returns more rows than it started with. Every sum is too large, and nothing raises an error.

**Count before, count after, and compare a total.**

No key today: every table is in the course repo.

### Today

| | |
|---|---|
| **12.1** | JOIN, many to one |
| **12.2** | LEFT JOIN, what does not match |
| **12.3** | The duplicate trap |
| **12.4** | Dates as text |
| **12.5** | LAG and ROW_NUMBER |
| **Then** | Two tests, Excel, Git, AI check |

# Today's database: seven tables

| Table | Rows | From |
|---|---|---|
| holdings | 200 | holdings.csv |
| issuers | 150 | issuers.csv |
| ratings | 18 | ratings.csv |
| benchmark | 821 | benchmark.csv |
| trades | 374 | trade_blotter.csv |
| treasury | 35,728 | the Treasury file, long form |
| spreads | 2,940 | synthetic spreads (invented) |

## The Treasury file in long form

```
date        tenor  years  yield_pct
2015-01-02  1 Mo   0.083  0.02
...
```

One yield column: the tenor becomes a value you pass as a ? parameter. Dates are ISO text.

**Control figure: 374 trades, 117,473,749.35 dollars of principal.**

# JOIN: many to one

```
SELECT h.cusip, i.issuer, r.grade, ...
FROM holdings AS h
JOIN issuers AS i ON h.ticker = i.ticker
JOIN ratings AS r ON h.rating = r.rating
```

JOIN alone is an inner join: a row with no match is dropped without a message. The count is what tells you.

**Trades by grade: 234 investment grade, 140 high yield; together, the blotter's 374.**

| Table | Rows | cusips | tickers |
|---|---|---|---|
| holdings | 200 | 200 | 115 |
| issuers | 150 | | 150 |

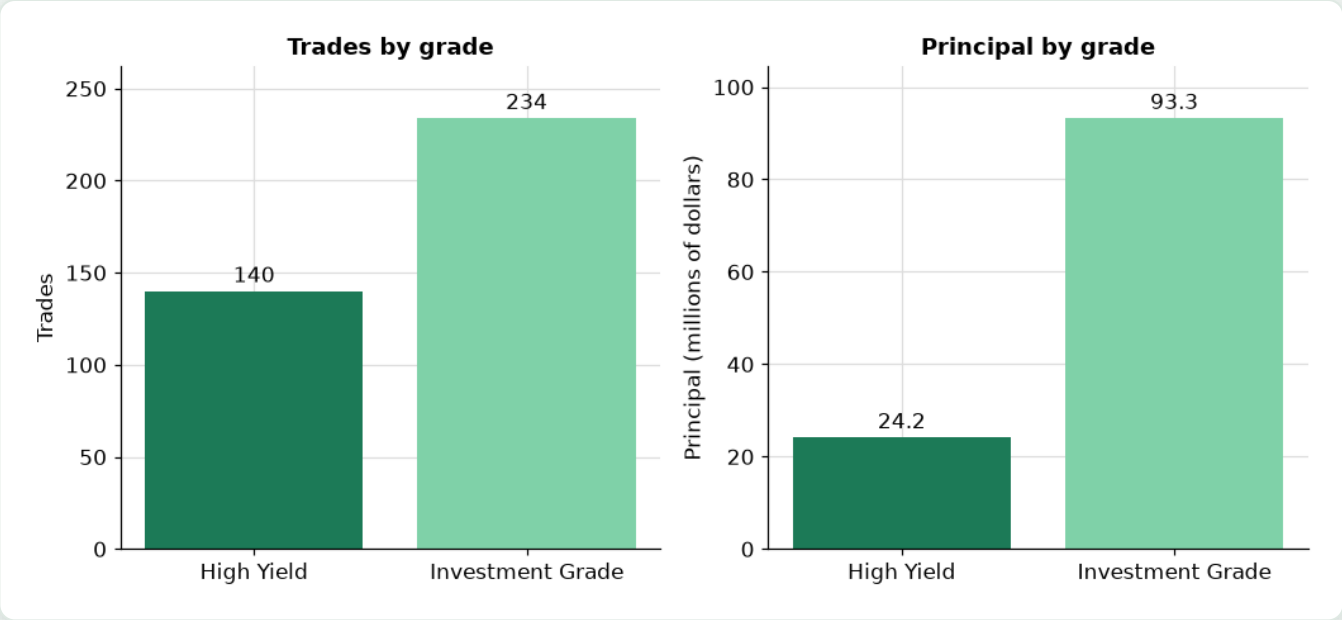
Many to one is safe only when the key is unique on the one side: ticker is unique in issuers, not in holdings.

```
merge(..., validate="many_to_one")
```

# Two joins, one control total

| Grade | Trades | Principal ($) |
|---|---|---|
| High Yield | 140 | 24,176,618.65 |
| Investment Grade | 234 | 93,297,130.70 |

Trades to holdings on cusip, holdings to ratings. The two grades add back to the blotter: 374 trades and 117,473,749.35 dollars. Neither join added or lost a row.




**Styled table: .format is Format Cells, .bar is Data Bars (Conditional Formatting).**

# LEFT JOIN: what does not match

```
SELECT b.cusip, b.ticker, b.weight_pct
FROM benchmark AS b
LEFT JOIN holdings AS h ON b.cusip = h.cusip
WHERE h.cusip IS NULL
```

**Nothing equals NULL, not even NULL: the test is IS NULL, never = NULL.**

```
merge(..., how="left", indicator=True)  # left_only
```

**621**

benchmark bonds not in the book

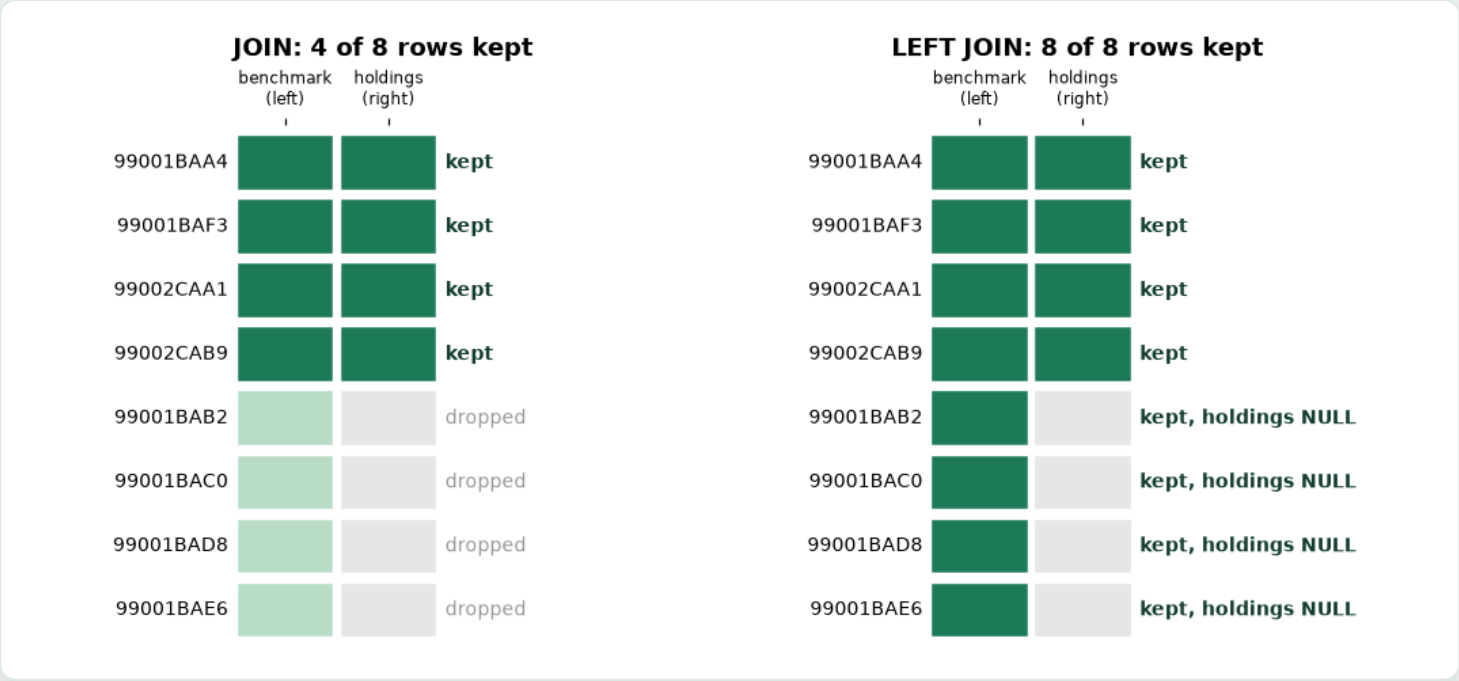
**200**

benchmark bonds the book holds

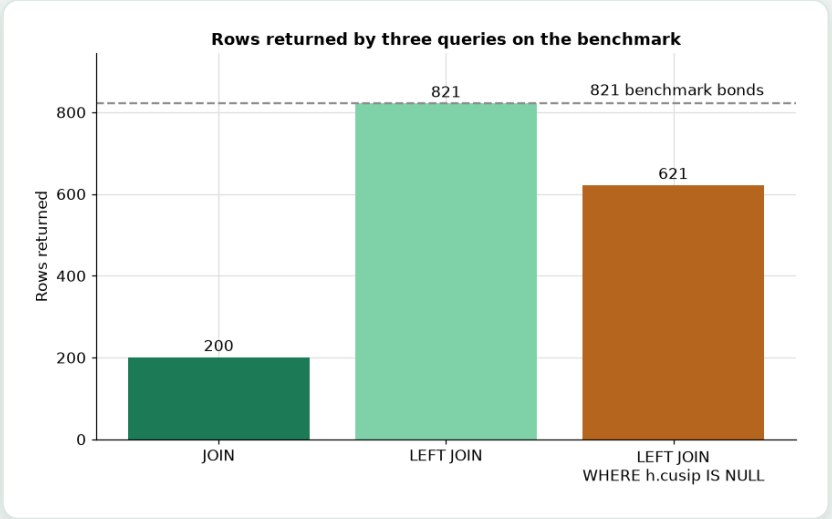
Their share of benchmark weight: 75.4 percent not held, 24.6 percent held.

**Coverage, described. Nothing about what the book should hold.**

# What JOIN and LEFT JOIN keep




Dark green: kept. JOIN drops the four bonds the book does not hold, without a word. LEFT JOIN keeps them, with NULL where the holdings columns would be.




200 from JOIN, 821 from LEFT JOIN, 621 with IS NULL.

# The duplicate trap

```
JOIN benchmark AS b ON h.ticker = b.ticker   -- trap
JOIN benchmark AS b ON h.cusip = b.cusip     -- fix
```

**200 to 1,213**

rows, joined on ticker

**2.99 bn**

market value after, from 490.8 mn

150 benchmark tickers have more than one bond, up to 8. Each holding matched every bond of its issuer.

## Two ways to catch it

**pandas**

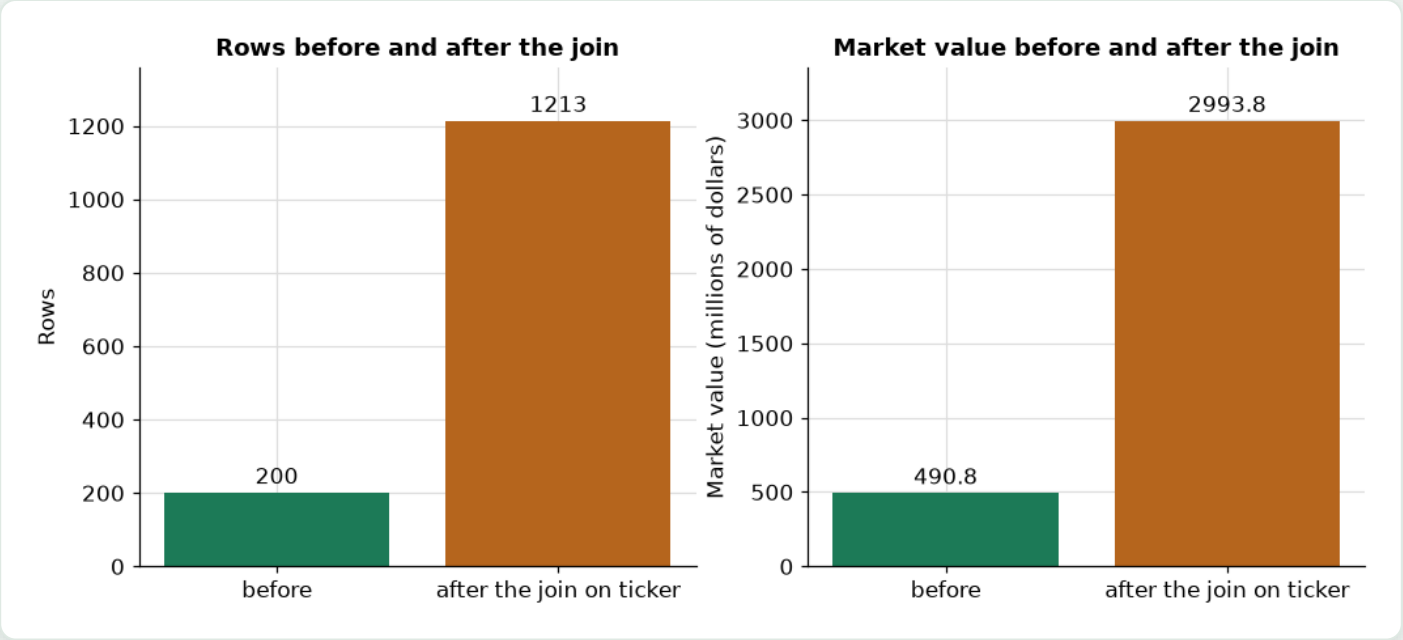
validate="many_to_one" raises MergeError before the merge is built.

**SQL**

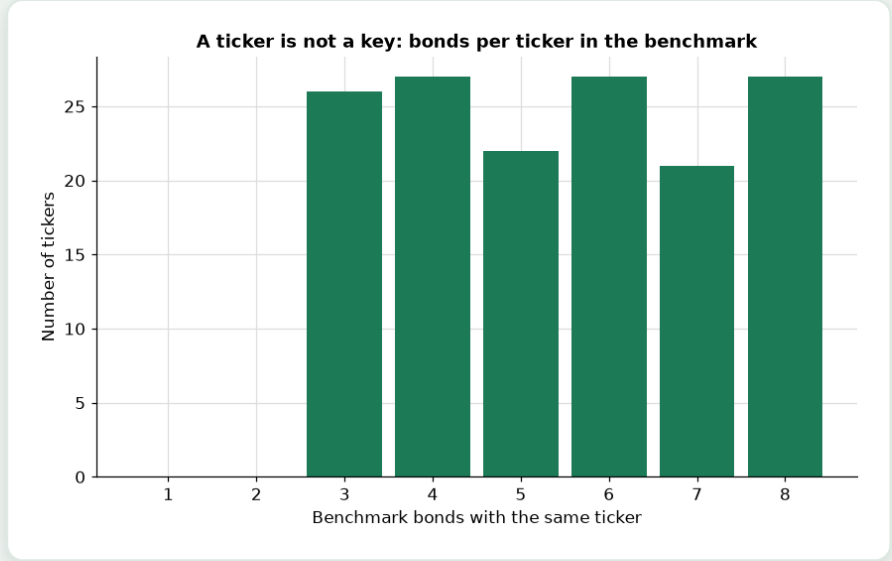
No such switch. Count before, count after, compare a total.

**Join on cusip: it names one bond.**

# The duplicate trap, drawn




200 rows became 1,213, and the market value grew with them. No bond was bought.




A ticker repeats: up to 8 bonds each. A cusip names one bond.

# Time series in SQL: dates as text

| Comparison as text | SQLite says | In date order? |
|---|---|---|
| `'2026-09-30' < '2026-10-02'` | 1 (true) | yes |
| `'09/30/2026' < '10/02/2025'` | 1 (true) | no: month first |

ISO text sorts in date order: year, month, day, each a fixed width. SQLite has no date type.

```
WHERE tenor = ? AND date BETWEEN ? AND ?
GROUP BY substr(date, 1, 7)        -- the month
JOIN spreads AS s ON t.trade_date = s.date
```

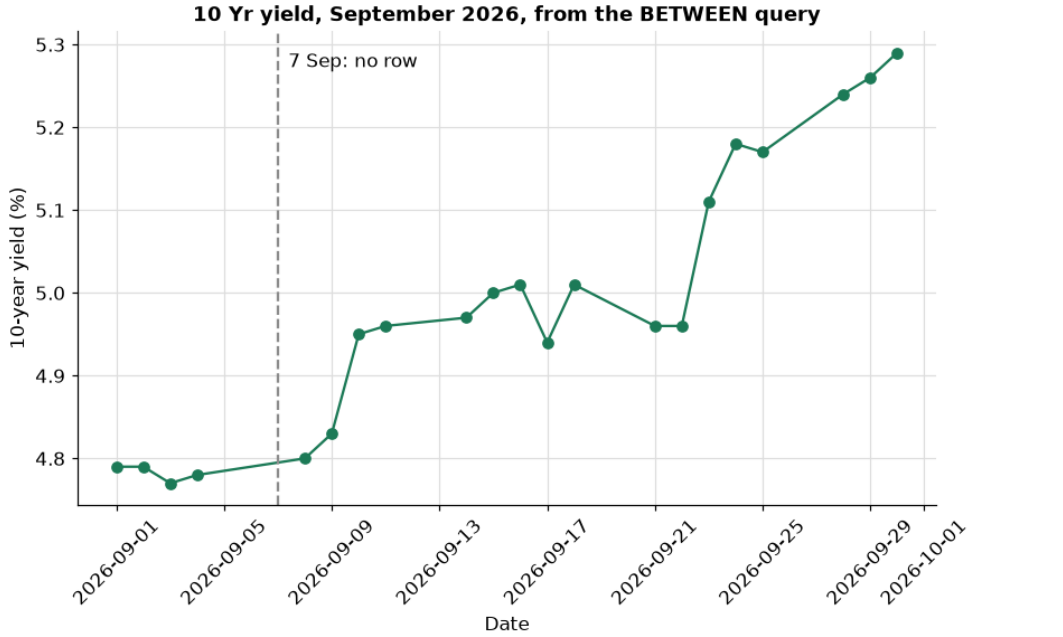
**21**

market days of the 10 Yr in September

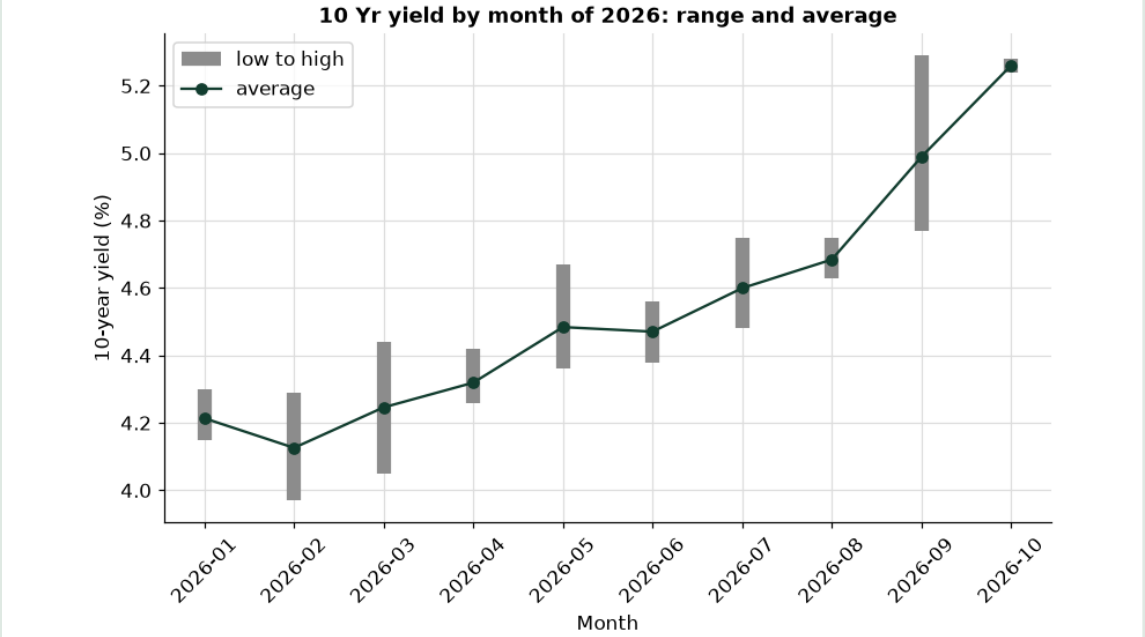
BETWEEN includes both ends, like a label slice in pandas. It works because the dates carry no time.

**Back in pandas: pd.to_datetime on the text column.**

# September and the months, drawn

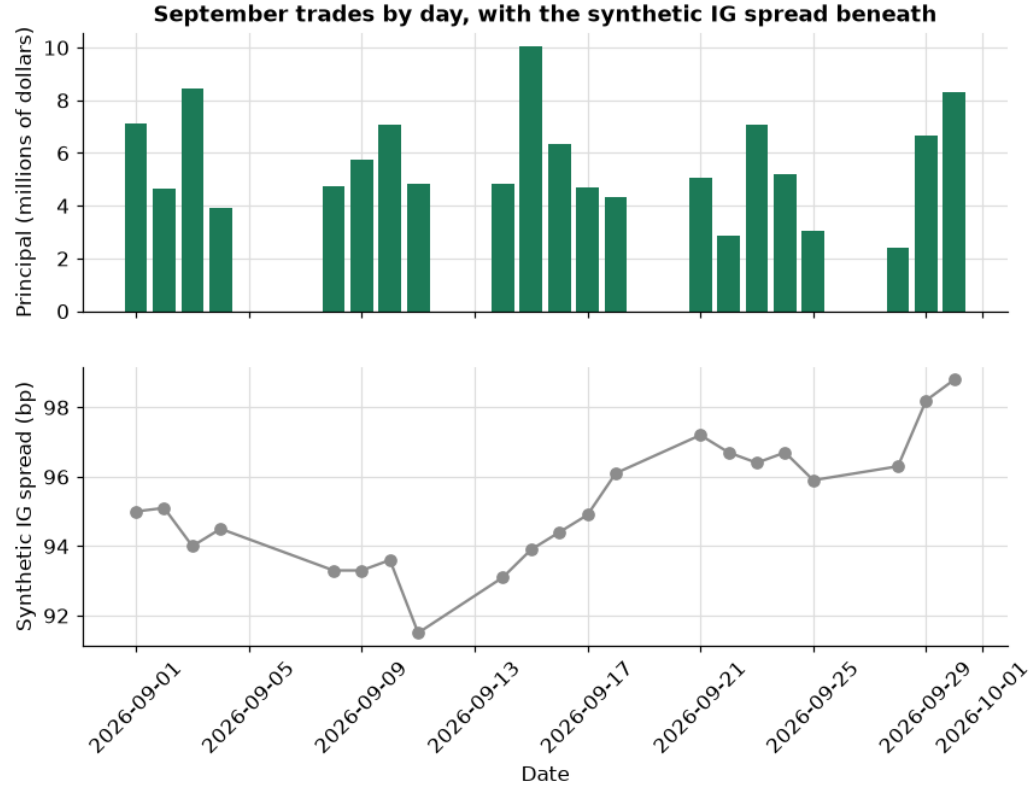



21 market days. The query returns only rows that exist, so the line crosses 7 September.




GROUP BY substr(date, 1, 7): MIN, MAX, and AVG for each month.

# A join on a date




| Month | Days | Low | High | Average |
|---|---|---|---|---|
| 2026-01 | 20 | 4.15 | 4.30 | 4.213 |
| 2026-02 | 19 | 3.97 | 4.29 | 4.126 |
| 2026-03 | 22 | 4.05 | 4.44 | 4.246 |
| 2026-04 | 22 | 4.26 | 4.42 | 4.319 |
| 2026-05 | 20 | 4.36 | 4.67 | 4.484 |
| 2026-06 | 21 | 4.38 | 4.56 | 4.470 |
| 2026-07 | 22 | 4.48 | 4.75 | 4.600 |
| 2026-08 | 21 | 4.63 | 4.75 | 4.684 |
| 2026-09 | 21 | 4.77 | 5.29 | 4.989 |
| 2026-10 | 2 | 5.24 | 5.28 | 5.260 |

Each trade date is matched to the spread row of its day. The spread is the course's invented series.

# Window functions: LAG and ROW_NUMBER

```
(yield_pct - LAG(yield_pct) OVER (ORDER BY date)) * 100
```

```
ROW_NUMBER() OVER (PARTITION BY substr(date, 1, 7)
                   ORDER BY date DESC)   -- keep 1
```

**WHERE runs before the window. Compute on the whole history, then filter.**

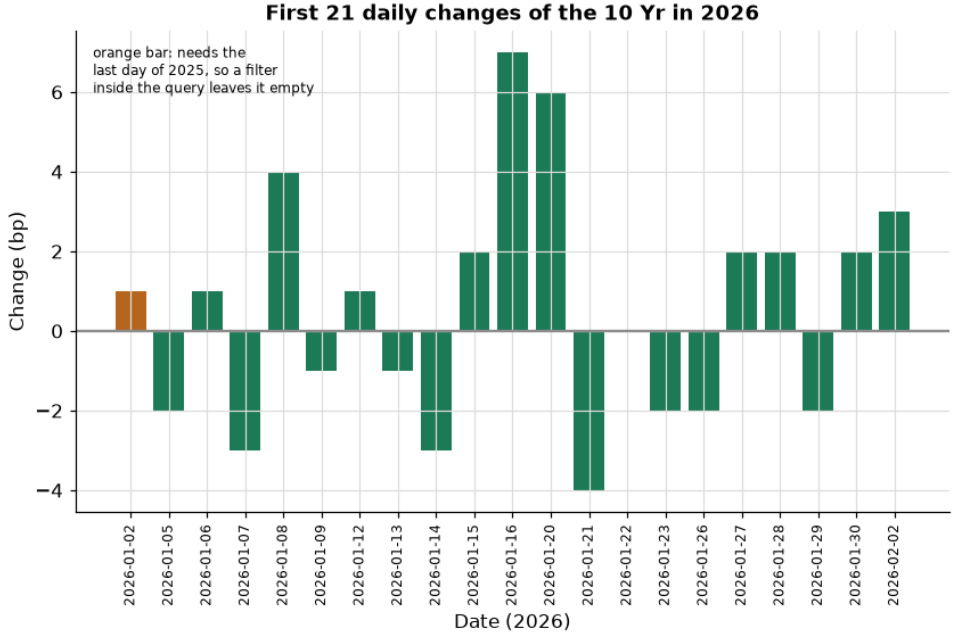
First change of 2026: filter inside, nan; filter outside, 1.0 bp.

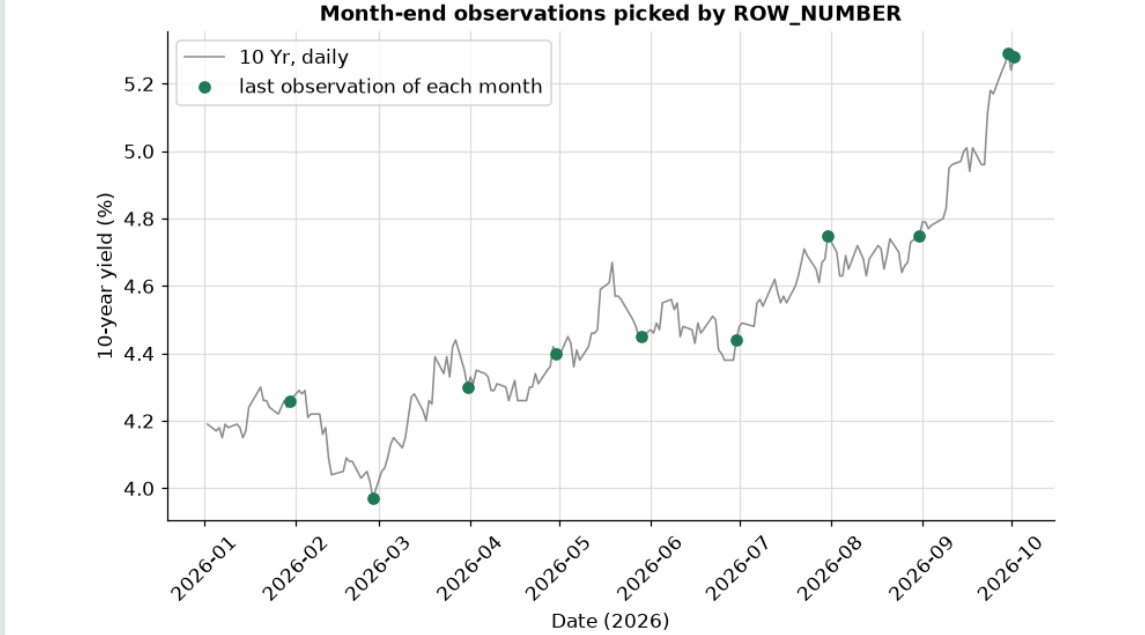
| SQL | Course function | Agrees on |
|---|---|---|
| LAG | change_bp | 2,940 rows |
| ROW_NUMBER | last_per_period | 142 months |

A window keeps every row and lets each one see its neighbours.

**Two tests: the join on cusip keeps every trade; month ends in SQL equal last_per_period. 48 passed.**

# The windows, drawn




The first change of 2026 needs the last day of 2025: 1.0 bp with the filter outside, empty with it inside.

ROW_NUMBER keeps the last row of each month: 142 months, all equal to last_per_period.

# Excel side by side

### XLOOKUP as a join

```
=XLOOKUP(F2,$A$2:$A$201,
         $C$2:$C$201)
```

By cusip: 0 #N/A over 374 trades. By ticker: the first match, so 178 trades get another bond.

### COUNTIF finds repeats

```
=COUNTIF($B$2:$B$201,B2)
```

Tickers: 145 rows above 1, from 60 tickers, up to 6. Cusips: largest 1, a key.

### Power Query Merge

Trades with holdings on cusip: 374 rows. Holdings with the benchmark on ticker: 1,213 rows; on cusip: 200. Left Anti, benchmark against holdings: 621.

Also tested: SUMIFS by side; MAXIFS with EOMONTH, then XLOOKUP, for month ends.

Tested in Excel 16.0 build 20430, 2026-10-04.

# Save the day: one file's history

### 1. Module

marketdata.py, the tests, and .gitignore with *.db

### 2. Joins

queries.sql with the day's joins

### 3. Time series

BETWEEN, LAG, and ROW_NUMBER added

```
git log --oneline -- queries.sql      # only the commits that touched the file
git show HEAD --stat                  # the last commit: message, files, lines
```

**When a query changed, and why: two commands answer it.**

# The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.

1

## Docstring

Units, conventions, and what it must never do: what you asked for.

→

2

## Code back

Written for the course in the style of an AI answer. Read it.

→

3

## Test first

Assert what the docstring promises. Run it: it fails.

→

4

## Fix

Find the rule it broke, fix it, run the test again.

**Today: trades_with_sector promises one row per trade, and joins trades to holdings on ticker.**

*Code that runs is not the same as code that is right.*

# What you can do now

- Join tables in SQL and in pandas, and prove they agree
- Find what does not match with LEFT JOIN ... IS NULL
- Count before and after every join; compare a total
- Dates as ISO text: BETWEEN, substr, joins on date
- LAG and ROW_NUMBER, filtering after the window

**Week 3, Day 3** Working with an AI assistant: the prompt as a spec, reviewing its code, and what never to paste